Improved Detection and Diagnosis of Faults in Deep Neural Networks Using Hierarchical and Explainable Classification

Sigma Jahan
Dalhousie University,
sigma.jahan@dal.ca

Mehil B Shah
Dalhousie University,
shahmehil@dal.ca

Parvez Mahbub
Dalhousie University,
parvezmrobin@dal.ca

Mohammad Masudur Rahman
Dalhousie University,
masud.rahman@dal.ca

***Abstract*—Deep Neural Networks (DNN) have found numerous applications in various domains, including fraud detection, medical diagnosis, facial recognition, and autonomous driving. However, DNN-based systems often suffer from reliability issues due to their inherent complexity and the stochastic nature of their underlying models. Unfortunately, existing techniques to detect faults in DNN programs are either limited by the types of faults (e.g., hyperparameter or layer) they support or the kind of information (e.g., dynamic or static) they use. As a result, they might fall short of comprehensively detecting and diagnosing the faults. In this paper, we present DEFault (Detect and Explain Fault) – a novel technique to detect and diagnose faults in DNN programs. It first captures dynamic (i.e., runtime) features during model training and leverages a hierarchical classification approach to detect all major fault categories from the literature. Then, it captures static features (e.g., layer types) from DNN programs and leverages explainable AI methods (e.g., SHAP) to narrow down the root cause of the fault. We train and evaluate DEFault on a large, diverse dataset of $\approx$ 14.5K DNN programs and further validate our technique using a benchmark dataset of 52 real-life faulty DNN programs. Our approach achieves $\approx$ 94% recall in detecting real-world faulty DNN programs and $\approx$ 63% recall in diagnosing the root causes of the faults, demonstrating 3.92%–11.54% higher performance than that of state-of-the-art techniques. Thus, DEFault has the potential to significantly improve the reliability of DNN programs by effectively detecting and diagnosing the faults.**

***Index Terms*—Deep Neural Networks, Dynamic Analysis, Model fault, Static Analysis, Training fault**

I. Introduction

In recent years, Deep Neural Network (DNN) [1] applications have been used in many critical domains, including but not limited to fraud detection [2], software debugging [3], [4], medical diagnosis [5], facial recognition [6], and autonomous driving [7]. Their widespread adoption marks a major shift towards Software Engineering 2.0 [8]. In this new paradigm, intelligent components powered by DNN models are integral to software systems, significantly influencing software development and maintenance practices [9]. However, despite the increasing popularity and success, DNN-based applications still suffer from reliability issues, and their faults are harder to detect and resolve compared to traditional software faults [10]. In traditional software systems, the control flows and data flows can be inspected and analyzed to reason about an underlying fault [11]. However, DNN models rely on high-dimensional tensors with millions or billions of parameters, where individual values may lack representation compared to the tensor as a whole [12]. Moreover, DNN models are stochastic and may yield different results across trials [13]. All these fundamental differences between traditional and DNN programs render traditional techniques ineffective [14] for detecting faults or errors in DNN programs.

In the context of DNN, training and model faults are most prevalent [10], [15], [16]. Training faults occur during the training process due to several issues such as gradient explosion, vanishing gradient, overfitting, and underfitting [16]–[18]. On the other hand, model faults arise from poor model architectures, such as incorrect or missing layer properties and flawed neuron connectivity [19].

To detect and diagnose faults in DNN programs, recent studies rely on two types of analysis, namely – static and dynamic analysis. Static analysis focuses on examining a DNN program's structure and syntax without execution [20], whereas dynamic analysis leverages runtime information captured during model training and inference [21]. Nikanjam *et al.* [22] propose NeuraLint, a static analysis-based technique that transforms the DNN program into a graph and employs predefined rules to detect faulty patterns. Zhang *et al.* [20] propose DEBAR that detects numerical faults from a DNN program based on several static analyses such as tensor abstraction, interval abstraction, and affine relation analysis.

Although static methods might be good for several fault types (e.g., API bugs, model bugs) [18], [22], they alone would fall short in capturing the intricate dynamics and runtime behaviors of DNN programs. Dynamic analysis-based approaches, such as UMLAUT [23] and AutoTrainer [24], utilize rule-based approaches to detect faults. Wardat *et al.* [25] propose DeepDiagnosis, a tree-based technique using predefined rules to guide developers from bug symptoms to likely causes. DeepLocalize [21] traces numerical errors back to the faulty layer using dynamic analysis, such as loss in the validation set. Despite the progress made by current approaches in detecting and diagnosing faults of DNN programs, they still suffer from several limitations, as follows.

(a) Lack of comprehensive support for root cause analysis of faults in DNN programs: According to Humbatova *et al.* [26], faults in DNN programs can be classified into seven

major categories (Table I) related to model construction and training. Existing techniques can analyze the root causes of only a subset of these categories of faults in the DNN programs. While some of them focus on model-related faults [22], others focus on training-related faults [23], [24], [27], [28], but rarely both. More importantly, none of the existing techniques supports weights or regularization-related faults, which encompass 8.3% of all DNN faults. Furthermore, the datasets constructed using the existing mutation frameworks [26], [29] might not be comprehensive and can represent only a subset of all possible faults. Consequently, techniques [27] based on them could identify only a subset of faults in the DNN programs. Therefore, the support from existing methods might not be comprehensive enough for root cause analysis of many faults in the DNN programs.

(b) Partial use of available information: Dynamic analysis could be effective in diagnosing certain training-related faults (e.g., incorrect learning rates and wrong loss functions). However, it alone is not sufficient for pinpointing the structural faults of a DNN model. Recent techniques [21], [23]–[25], [27] rely solely on dynamic analysis, overlooking the potential of static analysis in identifying model-related faults. Static analysis can capture the network architecture, layer configurations, and model parameters before training [14]. Thus, it can provide valuable insights for detecting structural faults where the existing techniques might fall short.

In this paper, we present a novel technique – DEFault (Detect and Explain Faults) – to detect and diagnose faults in DNN programs. It leverages both static and dynamic properties of DNN programs in a hierarchical classification framework to first detect their faults and then diagnose their root causes. Our solution can address the aforementioned challenges from existing works, which makes our work novel. First, DEFault can detect and explain model-related faults from DNN programs, offering a more comprehensive technique for fault localization. Second, it is trained on a more diverse and larger dataset than that of the state-of-the-art technique [27]. Unlike existing techniques that rely on either dynamic or static analysis for diagnosing faults, we have combined the best of both. Thus, our approach is better suited to detect and diagnose various types of faults in DNN programs.

We trained DEFault with our constructed dataset, which consists of 14,652 DNN programs (9,855 faulty + 4,797 correct) mutated from real-world Feed-Forward Neural Network (FFNN), Recurrent Neural Network (RNN), and Convolutional Neural Network (CNN). Then, we evaluated our technique on a benchmark dataset [27] containing 52 real-world DNN programs from StackOverflow (SO) and GitHub. We found that DEFault can identify faulty DNN models with 3.92% higher accuracy than the state-of-the-art technique [27]. It can also diagnose both training and model-related faults with 11.54% higher accuracy.

We thus make the following contribution in this paper:

(a) A novel hierarchical, explainable classification technique to detect faulty DNN programs and diagnose their root causes, leveraging static and dynamic analyses. An explainer module to explain the root causes of model-related faults using static features.
(b) A new, diverse dataset containing ≈ 14.5K DNN programs (≈ 9.5K faulty programs) targeting seven categories of faults in FFNN, RNN, and CNN.
(c) Proposed five novel dynamic features to predict faults in DNN programs, significantly improving fault prediction performance by ≈ 5.6% in terms of recall.
(d) Comprehensive evaluation of the proposed technique and comparison with multiple baselines [21], [23], [24], [27].
(e) A replication package containing our working prototype and experimental data for the replication [30].

II. Motivating Example

To demonstrate the effectiveness of our technique – DEFault, let us consider the example in Listing 1, taken from StackOverflow post #50079585. This faulty DNN program attempts to measure the severity of car crashes (minor, moderate, or severe) from input images. Unfortunately, it achieves an accuracy of ≈ 32%. According to the accepted answer on StackOverflow, the model has three faults. First, there should be exactly three neurons in the final `Dense` layer as there are three output classes. Second, the selected activation function `sigmoid` does not work well when the number of classes is more than two, and it should be replaced with `softmax` function. Finally, the selected loss function should be replaced by `categorical_crossentropy`, which is more appropriate for multi-class classification.

Listing 1 A Faulty DNN Program from StackOverflow

```
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=input_shape))
model.add(Activation('relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
...
model.add(Flatten())
model.add(Dense(64))
model.add(Activation('relu'))
model.add(Dropout(0.5))
model.add(Dense(1))
model.add(Activation('sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer='rmsprop',
              metrics=['accuracy'])
```

Among the existing techniques for fault detection, AutoTrainer [24] fails to detect any issues, probably due to its rule-based design, which only identifies patterns matching predefined symptoms for specific training problems – none of which were present in the motivating example. UMLAUT [23] incorrectly diagnoses the fault as 'missing activation functions' since its heuristics do not align with the actual program fault. DeepLocalize [21] fails to locate any fault as it is specifically designed to locate numerical errors (e.g., NaN or infinity in loss calculation) that were not present in the example model. Similarly, DeepFD [27] was not designed to handle layer-related faults and also fails to detect the loss function fault. In contrast, our technique, DEFault, integrates both static and dynamic analyses, enabling it to comprehensively detect and diagnose faults. Using dynamic features, DEFault detects the incorrect loss function. By leveraging static features with the

explainer module, our tool finds the inappropriate activation function (`sigmoid` instead of `softmax`) and the incorrect neuron count in the final Dense layer.

III. Background

In this section, we discuss the most relevant existing approaches and attempt to place our work in the literature. Existing approaches adopt the following methodologies: static analysis, dynamic analysis, mutation-based methods, and adversarial techniques.

Static analysis-based techniques detect faults or errors in the DNN programs by analyzing their structures and syntax without execution. Tools like NeuraLint [22] use graph transformations and predefined verification rules to identify design inefficiencies like incompatible loss functions or poorly configured layers. Similarly, NerdBug [31] adopts an abstraction-based approach to efficiently detect structural and coding bugs (e.g., API misuses and logic errors). DEBAR [20] employs tensor abstraction and interval analysis to detect numerical bugs in neural architectures.

Dynamic analysis-based techniques capture model behavior during model training, complementing static approaches. For instance, CRADLE [28] detects bugs in DL libraries through cross-backend testing and anomaly tracking. Tensfa [32] uses decision trees to classify crash messages and performs shape tracking and data dependency analysis to debug tensor shape faults. MODE [33] employs state differential analysis to identify faulty neurons causing misclassification, addressing overfitting and underfitting issues. DeepDiagnosis [25] maps anomalies in training metrics (e.g., vanishing/exploding gradients) to potential root causes using predefined rules. AutoTrainer [24] extends this by detecting and repairing common training issues like exploding gradients and slow convergence in real-time. DeepFD [27] uses a learning-based approach to classify five common training faults with high accuracy. DeepLocalize [21] locates numerical errors in faulty layers or hyperparameters by analyzing runtime values like weights, gradients, and loss. UMLAUT [23] provides debugging recommendations by heuristically mapping runtime symptoms to fault categories. GRIST [34] uses gradient backpropagation to diagnose numerical faults.

Mutation-based and *adversarial method-based* techniques detect faults in the DNN models by introducing controlled perturbations or modifications to models. DeepMuFL [35] employs mutation analysis and detects faulty mutants by ranking them based on their fault suspiciousness scores. Adversarial approaches like DeepFault [36] introduce perturbations to highlight model vulnerabilities, such as neuron-specific robustness issues.

Our proposed technique, DEFault, uses a *hybrid learning-based* approach that integrates static and dynamic analyses within a hierarchical classification framework, enabling comprehensive fault detection and diagnosis. DEFault also supports a wider range of faults (seven main categories out of eight from the DL fault categories), including those arising during model construction and training. Furthermore, by incorporating SHAP-based explanations, DEFault enhances interpretability, enabling precise diagnosis of faults in both structural and training contexts.

IV. DEFault

A. Collection of Faulty DNN Programs

Collecting a large number of correct and faulty DNN programs to train a fault detector is a major challenge. Thus, we developed a systematic approach to collect DNN programs from StackOverflow, inspired by prior works [27], [37]. We leverage the tagging system and content-based criteria of SO. The criteria include the recency of a question (last three years), the presence of accepted answers (at least 1), and the answer being of high quality (5 or higher upvotes) [38]. We limited our search to *TensorFlow* and *Keras* to align with popularity and recent literature [21], [25], [27]. After this step, we got 328 DNN programs. We then manually reviewed each DNN program and corresponding SO post containing a model structure (e.g., improper layer configurations) or training (e.g., overfitting) related fault. For example, the errors concerning import statements or API types might not be directly related to the training or model aspects of the DNN programs. These errors often stem from the environment setup, package versioning, or API usage rather than training steps or model structures. We also carefully examine the SO posts to verify that they contain sufficient information about faulty DNN models, including symptoms, faults, code snippets, and potential solutions. After all the filtration steps above, we got 89 DNN programs.

B. Reproduction and Repair of Faulty DNN Programs

After collecting 89 faulty DNN programs, we conducted another extensive manual analysis where we reproduced and repaired them. First, we determine the feasibility of reproducing each bug by executing the corresponding DNN programs. If the DNN program crashed due to API upgrades, versioning, or typo issues, we fixed them. Then, we execute the code and attempt to reproduce the symptoms of bugs reported at SO. After this step, we found 60 DNN programs reproducing the bugs, which were used for subsequent analysis. For each of the 60 reproducible faulty programs, we carefully studied the corresponding SO post, including the problem description, code snippets, and the accepted answer, to understand the root cause of the bug and the suggested fix. We then applied the fix to the faulty model and re-ran the code to verify whether the bug was successfully repaired. In cases where the suggested fix from the SO post did not completely resolve the issue, we further investigated the problem and performed additional modifications until the faulty program was fully repaired. After repairing each faulty model, we designed specific test cases based on the expected behavior described in the SO post. We then tested the fixed version with the provided dataset or a suitable alternative (e.g., dummy dataset). We compared the output of the repaired model with the expected results to confirm that the bug was resolved and that the model was functioning as intended. We successfully repaired all 60 faulty DNN programs in our dataset.

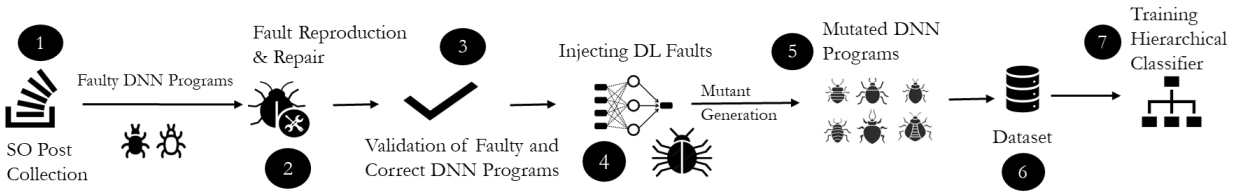

Fig. 1. Schematic diagram of dataset preparation and classifier model training

C. Validating Faulty and Correct Models

To validate the manual selection, filtration, reproduction, and repair of faulty DNN programs, two authors of this work independently reviewed the SO posts at each stage of the process. During the initial filtration stage, both authors examined all potential faults to determine their eligibility for inclusion in the study. They assessed whether each post met the criteria for advancement to subsequent stages. After completing their assessments, the authors discussed their findings to resolve any discrepancies and reach a consensus with 83.2% value of Cohen Kappa, indicating a strong level of agreement [39]. For the reproduction and repair phases, the first author and the second author both conducted the process, starting with 5 bugs from the dataset. After achieving an initial Cohen Kappa of 61.5%, they held two meetings to identify and resolve the main reasons for their disagreements. In the next round, they worked with 10 more bugs and achieved a Cohen Kappa of 82.8%, which is considered an almost perfect agreement [39]. Subsequently, the first author reproduced the remaining 45 bugs while the second author checked the reproduction, achieving an average Cohen Kappa of 85.4%.

D. Injecting Faults into DNN Programs

Due to the scarcity of real-world faulty DNN programs, we use mutation-based techniques to generate synthetic data [26]. This approach allows us to systematically introduce a wide variety of faults into the DNN programs. There have been a few frameworks to inject faults in DNN programs. DeepMutation [29] can inject several types of faults into the layers and weights of the DNN program. However, these fault types may not be representative of the diverse range of faults encountered across different architectures (e.g., FFNN, RNN, CNN). DeepCrime [26] is the state-of-the-art technique for fault injection that can inject six categories of faults in DNN programs. However, it cannot inject any fault into the layers of DNN models. Our manual analysis shows that 21.67% of DNN faults could be associated with the layer. Thus, DeepCrime might not be sufficient to generate mutant models containing layer-related faults. Moreover, it does not support the creation of mutants from RNNs. To address these limitations, we extended DeepCrime in two regards. First, we added ten mutation operators to inject faults in the layers of DNN models, following the guidelines from the modern taxonomy of faults in DNN programs [14]–[16], [26]. We apply these mutation operators by systematically selecting appropriate layers. For instance, to apply the *change activation function* operator, we randomly choose from the layers with an activation function and replace it with another function, randomly chosen from a set of 11 standard activation functions (e.g., ReLU, Sigmoid). Similarly, we apply regularization and architectural mutations directly to compatible layers to ensure the diversity of the mutated models. We carefully limit all mutation parameters within practical boundaries (e.g., CNN kernel sizes between 1–7) in accordance with the literature [40], [41] to maintain realistic and functional model configurations. Details can be found in our replication package [30].

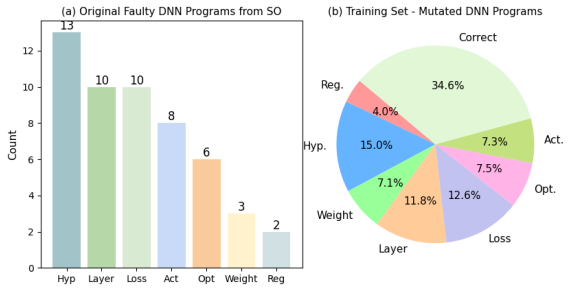

Fig. 2. Distribution of fault types in our datasets

This extension ensures comprehensive coverage of fault types in DNN programs. Then, we also add support for RNNs with all fault types. We use `isKilled()` from the DeepCrime method to determine if a mutated model is faulty. It works by comparing the mutant's accuracy distribution with that of the original model on a testing set. We use statistical tests (Generalized Linear Model [42]) to assess the significance and effect size (Cohen's d [43]) of the difference between the two accuracy distributions. If the mutant's accuracy distribution is significantly different and worse than the original model's, it is considered faulty and labeled accordingly; otherwise, it is considered correct.

Inspired by mutation testing research [26], [29], [44], we use the mutation score to check their applicability to different architectures (e.g., FFNN, CNN, RNN). The mutation score is defined as the proportion of mutation operator instances killed by the training set and the test set over all those killed by the training set. A high mutation score indicates that the new mutation operators effectively create meaningful faults that both the training and test sets can detect. The experiment shows high mutation scores (0.85 for FFNN, 0.83 for RNN, and 0.87 for CNN) for our mutant models. In real world, a single DNN program might have multiple faults. To create mutants with multiple faults, we adopt heuristics inspired by DeepFD [27] and inject an average of three and a maximum of seven types of faults in a single mutant (i.e., DEFault supports seven categories of faults). The process involves sequentially injecting faults one after another. We then validated the injected faults using the same technique (i.e., `isKilled()` method [45]). This approach ensures that all

TABLE I
FAULT INJECTION SUPPORTED BY DEFAULT AND DEEPCRIME

Fault Group	Hyperparameter	Layer	Loss	Activation	Optimization	Weights	Regularization	FNN	CNN	RNN
DEFault*	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
DeepCrime	✓	✗	✓	✓	✓	✓	✓	✓	✓	✗

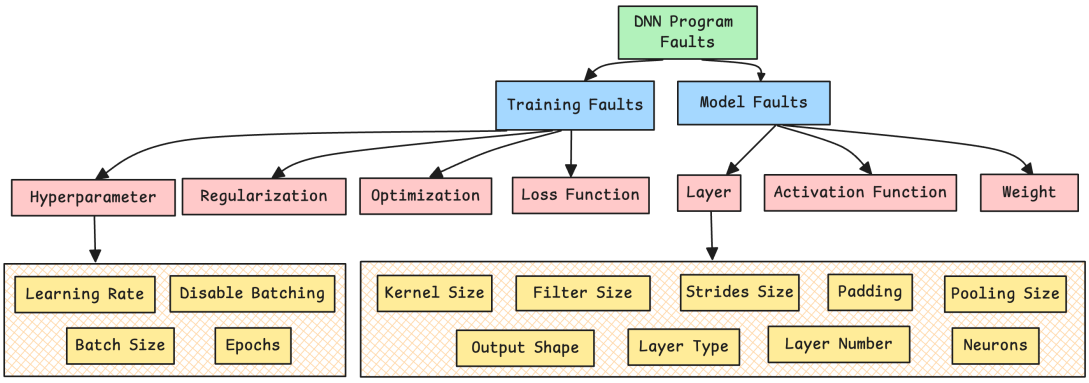

Fig. 3. Types of DNN faults and their root causes

mutations remain mathematically valid, operationally sound, and representative of real-world scenarios.

Table I presents the fault categories used for injecting faults into DNN programs. Due to the stochastic nature of DNN programs, we follow prior works [24], [27], [37] to reduce the randomness and retrain and evaluate each model 15 times. We focused on seven key fault categories from DeepCrime [26] (see Fig. 3), excluding data-centric mutation operators, as our work focuses on the faults encountered during model construction and training rather than from training data quality. Note that categories like Weights and Regularization affect DNN's training process globally [46], [47], making their further diagnosis redundant. In contrast, Layers and Hyperparameters have localized effects, making their fine-grained subcategories (e.g., filter size, learning rate) essential. Therefore, we further divide only Layer and Hyperparameter faults into their subcategories for effective root cause analysis.

E. Preparation of Training Dataset

We collect 60 DNN programs, apply the mutation operators (refer to Table I), and capture 14,652 mutants. We also apply killability criteria, and collect 9,855 faulty models and 4,797 correct models. During mutant creation, we capture their static and dynamic features as follows.

1) Static Feature Extraction: Static features are characteristics of a DNN model that can be extracted by analyzing the source code without executing the code [48]. These features provide insights into the structure, complexity, and design choices of the DNN model [22]. We collect five categories of static features as shown in Table II. All these features, proposed by the existing literature [15], [22], [49], showed their potential to identify faults in DNN programs. Furthermore, there exists a direct mapping between static features to specific DNN code sections, which could be faulty [50], [51]. As a result, these static features could effectively diagnose the root cause of a fault in the DNN model (see Section IV-G).

2) Dynamic Feature Extraction: Dynamic features, which are extracted during model training, can provide valuable

TABLE II
STATIC FEATURES EXTRACTED FROM DNN MODELS

Feature Components	Description
Layer Counts	Diversity. Count of different layer types and the unique type of layers (e.g., Dense, Conv2D, LSTM)
Neurons Count	Number of neurons in all applicable layers
Input Shapes	Dimension of the input layers
Output Shapes	Dimension of the output layers
Mismatch Detection	Detecting mismatches in input and output dimensions between consecutive layers

insights into the symptoms and root causes of faulty DNN models [21], [24], [25], [27], [28]. DEFault collects 23 runtime features (Table III), periodically captured at every epoch, using our custom callback methods implemented within each DNN program. Among them, 17 features were taken from existing literatures [21], [24], [27] and are proven effective in detecting faults in DNN programs. The remaining six are our novel features. The choice of these novel features is grounded in practical observations and validated by research in deep learning [52]–[54].

Activation: Activation saturation can limit a model's learning capacity [25]. To calculate the percentage of saturated activation values at each epoch, we collect activation statistics and compare them against predefined thresholds, determining if they are saturated at their minimum or maximum value. This helps identify potential learning bottlenecks caused by activation saturation.

Learning Rate: DEFault tracks the adjusted learning rate at each epoch when the learning rate is scheduled or adaptive. This feature helps categorize hyperparameter and optimizer faults that might prevent model convergence [53]. We collect the adjusted learning rate value at each epoch, considering any scheduling or adaptations applied during training.

Hardware Utilization: We track the hardware utilization metrics, including CPU, GPU, and memory usage. These hardware-specific metrics help us identify potential faults and inefficiencies [54]. Abnormal hardware utilization metrics can indicate inefficient memory management, including too much

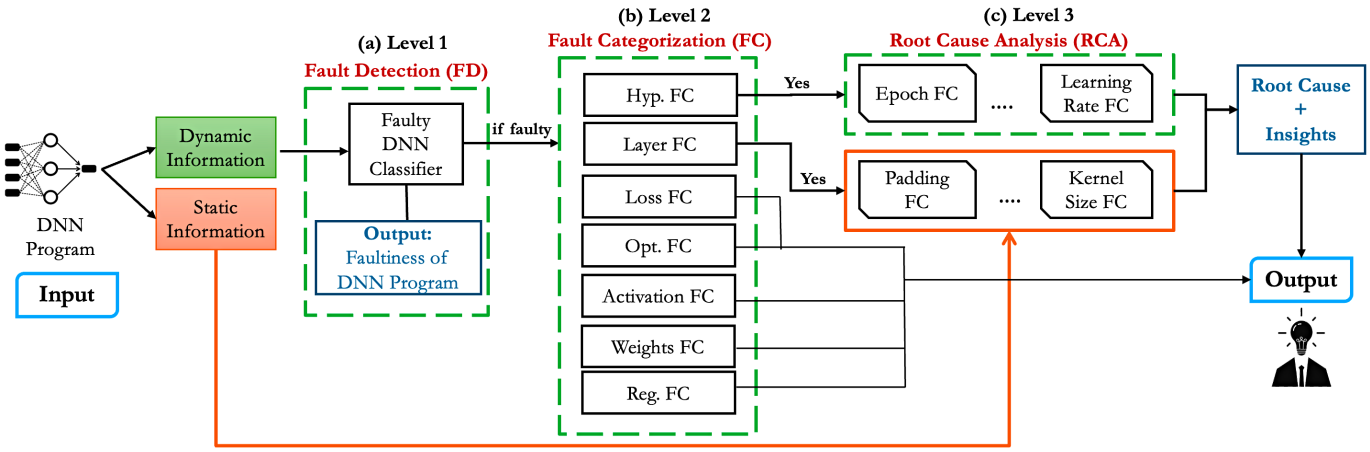

Fig. 4. Workflow of DEFault: (a) Fault detection, (b) Fault categorization, and (c) Root cause analysis

communication between CPU and GPU or unnecessary type conversions, leading to computational bottlenecks or even higher numerical error [55]. We measure these metrics regularly during training using system monitoring tools or APIs provided by the DNN framework (e.g., TensorFlow, Keras).

F. Training of Multi-Level Classifier

Once the training dataset is prepared, we train a hierarchical classification approach [56] on the dataset to detect and categorize faults in DNN programs. We choose the Random Forest [57] classifier as the base model due to its ability to handle high-dimensional data and capture non-linear relationships [58]. Our approach consists of multiple levels of classifiers. First, we identify whether a DNN program contains faults or not (see Section IV-H1). If yes, a set of binary classifiers identifies the specific fault category among the seven groups: Hyperparameter, Loss, Activation, Layer, Optimizer, Weight, and Regularization (see Section IV-H2). Since a single DNN program can have multiple faults (e.g., see Section II), we use seven binary classifiers to detect one or more fault categories. We do not use multilabel classifiers to avoid severe data imbalance problems. Among these categories, hyperparameter faults can emerge from different root causes. Thus, we use four binary classifiers to diagnose the root causes of hyperparameter faults (learning rate, disable batching, batch size, and epochs) at a finer-grained level.

We follow a similar training pipeline for all levels of classifiers. We load and preprocess the data (e.g., scaling, removing null values, encoding categorical values), split it into training, validation, and testing sets, initialize Random Forest classifiers, and train them on the preprocessed data. We also perform a grid search with 5-fold cross-validation to mitigate randomness and to find each classifier's best hyperparameters (e.g., max depth, max features) [59]. Grid search optimizes the model's performance by finding the best hyperparameters [60]. The dynamic features extracted from the DNN programs serve as input to these classifiers.

G. Construction of Explainer Module

Once fault categories are detected, we design an explainer module to diagnose their root causes. Since the developers act on the DNN model's structures (e.g., the number of neurons in a layer), we analyze the root cause of layer faults using static features. First, we train a new Random Forest model on the static features to detect whether a DNN program has a layer fault. Then, we use SHAP [61] to identify which static properties contributed the most towards the faulty verdict. SHAP can rank the training features based on their importance in a prediction [62]. It determines which static feature is the most likely to cause the specific layer fault.

H. Model Inference

Fig. 4 illustrates how DEFault identifies faulty DNN programs and analyzes their root causes. It uses our trained multi-level classifiers for inference. We use the DNN program from Listing 1 to describe DEFault's inference process.

1) Fault detection: At Level 1, we attempt to find out whether a given DNN program is faulty or not. First, we extract the dynamic information from the program, as described in Section IV-E2. Using the information, the Level 1 classifier predicts whether or not the DNN program has a fault. If the DNN program is faulty, we proceed to categorize the fault.

2) Fault categorization: At Level 2, we attempt to categorize the faults in the DNN program. We employ seven different classifiers that leverage dynamic features and target seven fault categories (Fig. 4). A DNN program can have one or more categories of faults. Each classifier predicts whether the DNN program has a fault from the corresponding category or not. For hyperparameter and layer categories, we continue with the root cause analysis as they are quite broad categories (check Fig. 3). For any other category, we stop the analysis.

3) Root Cause Analysis: We take two slightly different approaches to analyze the root cause of hyperparameter and layer faults. Hyperparameters of a DNN program are connected to the training phase and thus affect the training behaviors of the model. Therefore, we employ another set of binary classifiers leveraging the dynamic information to classify hyperparameter faults into more granular levels. These models identify whether the DNN program has faults in learning rate, disabled batching, batch size, or number of epochs.

On the other hand, the layers of a DNN program are often explicitly defined. Therefore, we employ an explainer module

TABLE III
DYNAMIC FEATURES EXTRACTED FROM DNN MODEL TRAINING

Runtime data	Monitored variables	Description
Training Metrics	loss, val_loss, train_acc, val_acc	Loss & accuracy on the training & validation sets
Weight	large_weight_count, cons_mean_weight_count, cons_std_weight_count, nan_weight_count	Statistics related to model weights, including large weights, constant mean/std, & NaN weights
Accuracy and Loss Trends	acc_gap_too_big, loss_oscillation, decrease_acc_count, increase_loss_count	Trends in accuracy and loss across epochs, including gaps, oscillations, and changes from previous epochs
Activation	dying_relu, activation_statistics*, saturated_activation*	Activation statistics (dying ReLUs, mean/std, saturated sigmoid/tanh)
Gradient	gradient_vanish, gradient_explode, nan_gradients_count, gradient_statistics	Gradient statistics (vanishing/exploding, NaNs, value distribution)
Learning Rate	adj_lr*	Adjusted learning rate for the epoch
Hardware Utilization	cpu_utilization*, gpu_memory_utilization*, memory_usage*	Hardware resource utilization: CPU, GPU memory, and overall memory usage.

*Proposed by DEFault

(SHAP) [61] to identify the static features (see Section IV-E1), potentially explaining the fault. In the case of the DNN program in Listing 1, we find that two static features from our explainer module, representing the number of `softmax` functions, contribute the most to mark the DNN program as faulty with layer fault. Since the faulty DNN program does not have any `softmax` activation function (i.e., count of softmax is 0), this implies that the model needs to have a softmax function. This way, we identify the root cause of a fault in a DNN program by following a series of inferences.

V. EXPERIMENT

We curate a large dataset of 14,652 mutated DNN programs and evaluate our technique DEFault – with standard performance metrics (e.g., accuracy, recall, precision, F1-score). To place our work in the literature, we compare our technique with four relevant baselines [21], [23], [24], [27]. We also assess the performance of DEFault in detecting and diagnosing 52 real-world faults in DNN programs. In our experiments, we thus answer four research questions as follows.

RQ_1: How does DEFault perform in detecting faulty DNN programs and categorizing their fault types?

RQ_2: How does the choice of features affect the performance of DEFault?

RQ_3: Can DEFault outperform the existing state-of-the-art techniques in DNN fault detection & categorization?

RQ_4: How does DEFault perform in diagnosing the root cause of DNN faults?

A. Experimental Dataset

Our training dataset consists of 14,652 mutated DNN programs (9,855 faulty & 4,797 correct). During the model training phase, we split our dataset as 70%-15%-15% for training, validation, and testing. Additionally, to evaluate the effectiveness of DEFault with real-world faults, we used a benchmark dataset [27] consisting of 52 real-world faulty models obtained from StackOverflow and GitHub.

B. Evaluation Criteria

To automatically evaluate our technique on the test data, we used four widely used performance metrics from ML classification, namely – accuracy, recall, precision, and F1-score [63]. In our experiment, we define true positive instances following existing studies [23], [24], [27]. A true positive instance for fault detection suggests that detecting the presence of a fault in the DNN program is accurate, regardless of its fault category. A true positive instance for fault categorization suggests identifying the category of the fault correctly (e.g., fault in hyperparameter), regardless of the root cause. For multiple fault types, if at least one fault type is accurately identified of a DNN program, it is considered a true positive [27]. Finally, a true positive instance for root cause analysis suggests identifying at least one of the root causes of the fault.

C. Replication of Baseline Techniques

In our research, we compared our proposed technique – DEFault – with four state-of-the-art baselines: UMLAUT [23], AutoTrainer [24], DeepLocalize [21], and DeepFD [27]. A detailed description of these methods is provided in Section III. To ensure a fair comparison, we directly used their official replication packages [21], [23], [24], [27] for the replication.

D. Evaluation of DEFault

*1) **Answering RQ_1** – Performance of DEFault:* In this experiment, we evaluate the performance of DEFault using appropriate performance metrics in both fault detection and fault categorization. Table IV shows the performance of DEFault in detecting faulty DNN programs.

TABLE IV
DEFAULT'S PERFORMANCE ON FAULT DETECTION

Class	Precision	Recall	F1-Score	Support
Faulty	0.97	0.98	0.98	1556
Correct	0.97	0.95	0.96	927
Macro Avg.	0.97	0.97	0.97	2483

Table V presents DEFault's performance in categorizing each type of fault in DNN programs. Loss, Optimization, and Activation faults achieved recalls of 0.96, 0.95, and 0.91, respectively. The high performance is attributed to the presence of dynamic features directly relevant to each of these fault types. Using the feature importance metric, we found that for Loss-related faults, dynamic features such as training loss and validation loss, loss oscillation, and increased loss count provide clear indicators. Similarly, Optimization faults benefit from features like gradient vanishing, exploding, NaN gradients, and adjusted learning rate [64]. Activation faults can be

effectively diagnosed using several dynamic features such as dying ReLU, saturated activation, and activation statistics [65]. The direct relevance of these fault-specific features enables DEFault to accurately identify issues of these three categories.

TABLE V
DEFAULT'S PERFORMANCE ON FAULT CATEGORIZATION

Category	Testset	Accuracy	Precision	Recall	F1-Score
Loss	555	0.97	0.97	0.96	0.96
Optimization	330	0.94	0.93	0.95	0.94
Activation	322	0.91	0.92	0.91	0.91
Hyperparameter	658	0.93	0.93	0.92	0.92
Layer	520	0.86	0.89	0.84	0.86
Regularization	176	0.95	0.94	0.95	0.94
Weights	312	0.90	0.90	0.89	0.89
Overall		**0.92**	**0.93**	**0.92**	**0.92**

Precision, Recall and F1-Score are weighted averages

In the Hyperparameter and Regularization fault categories, DEFault obtains a recall of 0.92 & 0.95. Based on feature importance, we found that dynamic features like GPU & CPU utilization, training metrics (loss, accuracy), gradient statistics, and adjusted learning rate are the most relevant features, which help to detect these training-related fault categories. For Layer and Weights faults, DEFault achieves a recall of 0.84 & 0.89, slightly lower than other fault types. Diagnosing these faults might be more challenging due to their model-related nature, the complexity of layer interactions, silent failures, non-linear effects, multiple problematic layers, and dependencies on initialization [15], [16], [22]. Moreover, DEFault achieves high precision values ranging from 0.89 to 0.97 across all fault categories, indicating that when DEFault identifies a fault and categorizes it, it is highly likely to be accurate in its assessment. The results are consistent across both test and validation sets, demonstrating the robustness of DEFault's fault detection and categorization capabilities.

Overall, DEFault demonstrates strong performance in fault detection and categorization across all fault types. As shown in Table V, DEFault achieves an overall accuracy of 92%, precision of 93%, recall of 92%, and F1-score of 92%. This means that DEFault accurately detected and correctly categorized 92% of the faults present in the dataset. The high precision indicates that when DEFault identifies a fault, it is highly likely to be correct, while the high recall shows its effectiveness in capturing most of the actual faults. These results highlight DEFault's robust overall capability in automating the fault diagnosis process in DNN programs.

TABLE VI
DEFAULT'S PERFORMANCE IN ROOT CAUSE ANALYSIS OF HYPERPARAMETER FAULTS

Root Causes	Testset	Accuracy	Precision	Recall	F1-Score
Batch Size	210	0.96	0.97	0.92	0.94
Learning Rate	195	0.97	0.96	0.96	0.96
Disable Batching	118	0.92	0.95	0.89	0.92
Epoch	135	0.94	0.93	0.92	0.92

*2) **Answering RQ_2** – Impact of novel and existing features on DEFault's performance:* We extracted five novel dynamic features from the runtime of DNN programs. To determine the impact of these factors on fault detection, we constructed a Generalized Linear Model [66]. It allows us to test the statistical significance of multiple independent factors in our technique. Moreover, using the global explanation of SHAP, where the absolute Shapley values of all instances in the dataset are averaged [67], we found the top five impactful features that drive the model's decisions in determining faulty or correct DNN programs. From Fig. 5, we notice a significant contribution of our newly added features, particularly GPU Memory Utilization and Adjusted Learning Rate. GPU Memory Utilization captures unusual tensor operations and data transfer patterns [68], indicating potential inefficiencies or errors, while Adjusted Learning Rate reflects dynamic changes during training, which are critical for identifying unstable training processes and optimization issues [69]. To further assess the impact of these new features, we conducted an ablation study by removing them from the feature set of our Level 1 classifier (i.e., fault detection). The fault detection performance decreased from 97% to 93% (accuracy), confirming the contribution of these five dynamic features to the overall performance of DEFault.

In our approach, we extracted two types of features: dynamic features from the training step (Table III) and static features from the source code (Table II). To determine which feature type is more effective for fault detection and categorization, we trained our classification models on each type individually. We found that static features alone are not particularly helpful in detecting faulty DNN programs. Using static features only, DEFault achieves 62.52% accuracy on the test set, whereas using dynamic features, the accuracy is 97%. Our experiment on incorporating both dynamic and static features did not improve the performance of fault detection. Rather it increased the computational overhead (e.g., processing time). Therefore, we excluded static features from the fault detection and categorization phase. However, we found static features to be crucial for fault diagnosis, particularly in conducting root cause analysis. Therefore, we incorporated these features into the explainer module (Section IV-H3).

TABLE VII
PERFORMANCE COMPARISON BETWEEN DEFAULT & BASELINES

Metrics	UM		AT		DLC		DFD		DEF	
	FD	FC	FD	FC	FD	FC	FD	FC	FD	FC
Test Set										
Accuracy	0.96	0.52	0.56	0.35	0.75	0.21	0.94	0.76	0.97	0.92
Precision	0.95	0.54	0.60	0.37	0.74	0.23	0.94	0.75	0.97	0.92
Recall	0.94	0.51	0.54	0.34	0.72	0.20	0.92	0.77	0.98	0.92
F1-Score	0.95	0.52	0.57	0.35	0.73	0.21	0.93	0.76	0.97	0.92
Benchmark										
Accuracy	0.91	0.27	0.23	0.19	0.69	0.13	0.90	0.52	0.94	0.63
Precision	0.89	0.28	0.25	0.20	0.69	0.15	0.90	0.52	0.93	0.63
Recall	0.90	0.26	0.22	0.18	0.67	0.12	0.88	0.53	0.94	0.63
F1-Score	0.89	0.27	0.23	0.19	0.68	0.13	0.89	0.52	0.93	0.63

FD: Fault Detection, **FC**: Fault Categorization, **UM**: UMLAUT, **AT**: AutoTrainer, **DLC**: DeepLocalize, **DFD**: DeepFD, **DEF**: DEFault

*3) **Answering RQ_3** – Comparison with existing baseline techniques:* We compare DEFault in fault detection and categorization with four established baselines: UMLAUT [23], AutoTrainer [24], DeepLocalize [21], and DeepFD [27]. Table VII shows that DEFault demonstrates better performance

compared to the existing state-of-the-art techniques in both fault detection and categorization tasks. On the testset, DEFault achieves a fault detection accuracy of 97.00% and a fault categorization accuracy of 92.20%, outperforming all other techniques. The closest competitor, DeepFD, attains 94.00% detection accuracy and 76.00% categorization accuracy. The performance gap is even more significant in real-world evaluation (benchmark), where DEFault has a 94.30% detection accuracy and 63.46% diagnosis accuracy. The other baseline techniques exhibit varying degrees of effectiveness. UMLAUT achieves high detection accuracy but struggles with fault diagnosis. However, AutoTrainer and DeepLocalize perform poorly in both detection and diagnosis tasks across both datasets. The limitations of these baseline techniques can be attributed to their approach and scope. UMLAUT, being a heuristic-based framework with specific faults (e.g., learning rate out of common range), may not generalize well to diverse fault scenarios. AutoTrainer focuses on a limited set of training problems, which may not cover all possible fault types. DeepLocalize is limited only to numerical error. In contrast, DEFault demonstrates superior generalization and adaptability by adopting a hierarchical classification framework that learns from a diverse dataset covering all training and model-related faults and leverages dynamic features to detect and categorize faults.

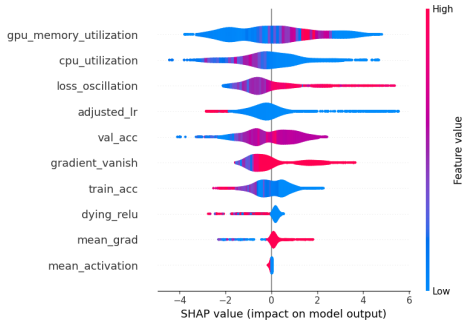

Fig. 5. Feature Impact on Faulty DNN Program Detection using SHAP

TABLE VIII
ROOT CAUSE ANALYSIS OF DEFAULT & BASELINE TECHNIQUES

#	Fault	UM			AT			DLC			DFD			DEF		
		FD	FC	RCA	FD	FC	RCA	FD	FC	RCA	FD	FC	RCA	FD	FC	RCA
1	act, lay	✓	✗	✗	✓	✓	✗	✓	✓	✗	✓	✓	✗	✓	✓	✓
2	loss, lay	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✗
3	act, lay	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✓	✓
4	ep, lay	✓	✓	✗	✓	✓	✗	✗	✗	✗	✓	✓	✓	✓	✓	✓
5	lay, act, loss	✓	✓	✓	✗	✗	✗	✓	✓	✗	✓	✓	✗	✓	✓	✓
6	opt, loss, lay	✓	✗	✗	✓	✗	✗	✓	✗	✗	✓	✓	✗	✓	✓	✓
7	ep, lr, act, loss, lay	✓	✓	✗	✓	✓	✗	✗	✗	✗	✓	✗	✗	✓	✓	✓

UM: UMLAUT, **AT**: AutoTrainer, **DLC**: DeepLocalize, **DFD**: DeepFD, **DEF**: DEFault; **FD**: Fault Detection, **FC**: Fault Categorization, **RCA**: Root Cause Analysis, **lr**: learning rate, **act**: activation, **opt**: optimization, **ep**: epoch, **bs**: batch size, **lay**: layer

*4) **Answering RQ_4** – Assessment of Root Cause Analysis:* To evaluate DEFault's ability to identify the root causes of hyperparameter-related faults (e.g., incorrect batch size), we measured its performance using accuracy, precision, recall, and F1-score. Table VI shows that DEFault performs well in identifying the root causes of hyperparameter-related faults. With accuracy, precision, recall, and F1-scores consistently above 0.89 for all four hyperparameter fault types (batch size, learning rate, disable batching, and epoch), DEFault demonstrates its effectiveness in pinpointing the specific root causes responsible for the faults.

Secondly, in our explainer module (Fig. 4), we focused on the root causes of layer-related faults and utilized the SHAP framework for local explanation [70]. From our benchmark dataset of 52 real-life faulty DNN programs, we collected a total of 7 layer-related faults for our assessment of DEFault. For each of the 7 layer-related faults, we used SHAP to identify the most important features contributing to the detection of faulty behavior. SHAP assigned importance scores to each static feature, allowing us to rank them based on their relevance to the fault. We then compared the top-ranked features identified by SHAP with the ground truth. To assess the performance of our explainer module, we calculated the Top@1 and Top@5 accuracy. These metrics measure the percentage of cases where the correct root cause feature is identified within the top 1 and 5 features ranked by SHAP, respectively. We observed that our explainer module achieved a Top@1 accuracy of 57.10% and a Top@5 accuracy of 85.71%. It means for 6 out of the 7 layer faults, the correct root cause feature was present within the top 5 features ranked by SHAP (see Table VIII). SHAP's instance-level explanations enable DEFault to effectively identify the most influential features contributing to layer-related faults, making it suitable for our benchmark dataset, which contains a limited number of such faults.

VI. CASE STUDY: EVALUATION OF DEFAULT

To demonstrate DEFault's applicability to real-world, complex DNN programs, we conducted a case study using the PixelCNN model from the *Sarus repository* at GitHub [71]. PixelCNN's intricate architecture, resembling real-world applications, makes it a suitable candidate for our case study. We evaluated DEFault using this model on the MNIST [72], a widely used dataset in DL [73], and report our results below.

A. Description of Fault

The PixelCNN program contained three types of faults in its version `a83c151` [71].

(a) Loss Function Fault: PixelCNN incorrectly used the `SparseCategoricalCrossentropy` loss function, which did not align with its objective.

(b) Hyperparameter Fault: PixelCNN used an insufficient number of training epochs (i.e., n=10 instead of the optimal n=75), which adversely affected the model's convergence and learning progress.

(c) Layer Faults: The model had misconfigurations in the hidden layer dimensions and residual block counts, which negatively affected its architecture and layers. In particular, the model suffered from these two layer-related issues as follows.

(i) *Root Cause 1 – Neuron Count:* The `hidden_dim` parameter was set incorrectly (i.e., n = 32 instead of 64). A reduced number of neurons in the layers restricted the model's capacity to capture intricate features within the data.

(ii) *Root Cause 2 – Layer Number:* The `n_res` parameter, the number of residual blocks in the model, was incorrect

(i.e., $n = 3$ instead of 6). Fewer residual blocks decreased the network's depth and representational power.

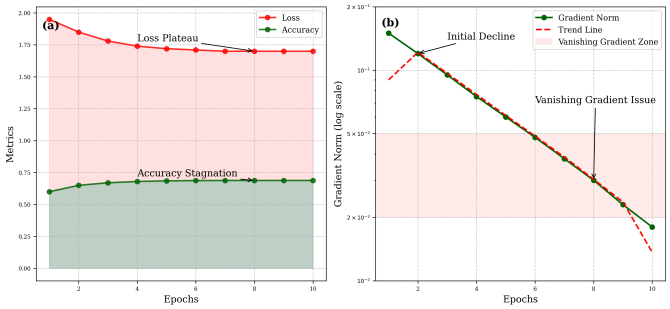

Fig. 6. (a) Loss-Accuracy (b) Gradient Norms during PixelCNN training

B. DEFault's Effectiveness & Result Discussions

Our proposed technique, DEFault, leverages both static and dynamic information and detects and diagnoses faults in the PixelCNN model in three steps as follows.

a) Step 1: Fault Detection: DEFault captures dynamic information from the PixelCNN model during its training using our custom callback functions. It monitors runtime features such as loss trends, gradients, and activation statistics. Based on anomalies detected in these metrics (e.g., stagnant loss trends, dead neurons, slow convergence, vanishing gradient), DEFault identified that the DNN program/model (i.e., PixelCNN) contained one or more faults.

b) Step 2: Fault Categorization: Upon marking the PixelCNN model as faulty, DEFault attempts to categorize its faults using seven binary classifiers. It leverages dynamic features collected from our custom callbacks during training. According to our experiments, DEFault successfully identified the presence of three faults: Loss Function Fault, Hyperparameter Fault, and Layer Fault in the PixelCNN model.

c) Step 3: Root Cause Analysis: DEFault also analyzes the root causes of the detected faults, leveraging dynamic and static features as follows.

Step 3.1: Hyperparameter Fault: DEFault captures several dynamic features, such as loss and accuracy trends, from the PixelCNN model during its training and analyzes their importance or relevance to the hyperparameter faults. Using the Level 3 classifier, our technique also marks the *number of epochs* as the root cause for the hyperparameter faults. Our post-hoc analysis (Fig. 6a) suggests that the training loss decreased slightly during the initial epochs but then stagnated. DEFault's continuous monitoring of loss values within an epoch and across multiple epochs might have helped capture the stagnated loss trends and identify the loss function fault (Level 2 classifier, Step 2). We also notice the stagnant accuracy throughout the training process of the PixelCNN model, suggesting incomplete model convergence. By capturing these training behaviors (e.g., stagnant loss and accuracy) at regular intervals, Default's Level 3 classifier confirms the presence of an epoch fault. Given the lack of model convergence, the number of epochs might not be sufficient, which explains DEFault's root cause behind the hyperparameter faults.

Step 3.2: Layer Fault: DEFault captures several dynamic features, such as activation and gradients, from the PixelCNN model during its training and analyzes their importance and relevance to the layer faults. DEFault captures static features (e.g., layer numbers, neuron count) from the PixelCNN model to diagnose the layer faults detected above. It conducts a SHAP-based analysis of the static features leveraging its Explainer Module (see Section IV-G) and generates the following explanations of the root causes.

Top@1 CountDense: Check the configuration and number of Dense layers.

Top@2 Max_Neurons: Verify the maximum number of neurons in any single layer.

Top@3 CountConv2D: Inspect the configuration of 2D convolutional layers.

All these messages suggest the number of hidden layers and the number of neurons as the potential root cause of the layer fault. Our post-hoc analysis suggests that during the training of the PixelCNN model, many neurons remained inactive with outputs at zero, indicating a higher percentage of 'dead neurons'. We also observed abnormally low gradient norms (see Fig. 6b), suggesting vanishing gradients in the model. DEFault's continuous monitoring of activation and gradient statistics within an epoch and across multiple epochs might have helped it capture activation saturation or vanishing gradients and identify the layer faults (Level 2 classifier, Step 2). We also noticed that static features such as `CountDense`, `Max_Neurons`, and `CountConv2D` are the most important in the explainer module targeting the layer faults. This also indicates that misconfigurations in the layers and neurons might have triggered the bug, which explains DEFault's identification of these aspects as the root cause behind the layer faults.

C. Limitations of DEFault

Although DEFault correctly identified three types of faults in the PixelCNN model using its hierarchical classifier, it misclassified an optimization fault. Fig. 6a shows that the faulty loss function in PixelCNN negatively affects the optimization process, resulting in stagnant loss and accuracy trends. Since optimization parameters are linked to loss calculation [74], which is faulty in PixelCNN, DEFault might have predicted the faulty optimization inaccurately. To understand this misclassification, we analyzed the top features influencing DEEault's loss and optimization classifiers. We found that features like `gpu/cpu_utilization`, `train_acc`, and `memory_usage` are influential in both classifiers, suggesting their overlapping behavior. Furthermore, our analysis reveals a significant correlation among their features. For instance, the negative correlation between `val_acc` and `adj_lr` (-0.95) highlights a strong connection between loss/accuracy calculation and the optimization process. Thus, DEFault might have difficulty distinguishing between loss and optimization faults due to their overlapping and strongly connected behaviors.

D. Comparison with Baseline Techniques

To validate DEFault's effectiveness in fault detection and diagnosis, we compared it with four baseline techniques.

DeepFD detected the hyperparameter fault and loss function fault but missed the layer fault. This limitation could be attributed to its focus on anomalies in specific dynamic features, neglecting activation-related dynamic features, which our experiments found to be relevant for identifying layer faults. Additionally, its pre-trained model lacks support for layer faults and does not account for architectural properties (e.g., static features), further contributing to this shortcoming. Moreover, DeepFD also misclassified optimization as a fault like DEFault, reinforcing our observation regarding the overlap between loss calculation and optimization.

UMLAUT relies on heuristics (e.g., validation accuracy trends) to map between symptoms and root causes of DNN faults. Using the heuristics, it was able to detect hyperparameter faults, but it could not pinpoint the root cause of the fault (e.g., number of epochs). It also failed to detect the loss function fault and layer faults, suggesting the limitations of heuristics against structural issues in the DNN model.

Autotrainer captures dynamic metrics from a DNN model during training and uses the rule-based technique to detect faults and repair the model. While it correctly identified the oscillating loss behavior caused by the incorrect loss function, it could not detect the hyperparameter fault or layer faults. This is likely because its rule-based design only looks for patterns that match the predefined symptoms for specific training problems. Hyperparameter faults, like insufficient epochs and layer faults, may not directly match these predefined patterns, which limits Autotrainer's ability to identify issues that it was not designed for.

DeepLocalize was designed to detect numerical errors like NaN or infinity in loss or activations, which were absent in the PixelCNN faults. Thus, it was not able to detect any of the three bugs from the PixcelCNN model.

Overall, DEFault demonstrated broader fault coverage by successfully identifying both runtime (e.g., loss function fault, hyperparameter fault) and structural issues (e.g., layer faults) in the PixelCNN model where the contemporary baselines fall short due to their limited scopes or specific focus areas.

VII. Threats To Validity

Threats of *internal validity* relate to experimental errors. Re-implementation of the existing baseline techniques could pose a threat. Since we used the replication packages provided by the original authors for all our baseline models, such a threat might be mitigated. We also repeat our experiments 15 times and compare the performance with that of baselines to mitigate any bias due to random trials [75].

Threats to construct validity [76] relate to the subjectivity in manual assessment. Filtration and reproduction of DNN programs during benchmark construction could pose such a threat. To minimize individual biases, two authors independently examined each subject, and we see a high agreement level between them (e.g., 85.4% Cohen Kappa score).

Threats to *external validity* relate to the generalizability of our work [77]. Whether the dataset represents real-world faults or not is such a threat. Since we collected all DNN-related posts from StackOverflow and filtered them through a rigorous set of criteria following existing literature, the threat is minimal. Whether the selected performance metrics measure the research problem properly or not is another threat. The literature well-defines all of our performance metrics, and the selection is supported by the baseline literature. Thus, the threat to external validity is also minimal.

VIII. Related Work

Mutation Testing: To overcome the scarcity of publicly available faulty DNN programs, prior works [27] have used synthetic models (i.e., mutant models) generated by DeepCrime [26] as the source of the training dataset. However, DeepCrime might be limited since it supports six out of seven major categories from the literature [15], [16]. It also does not support RNN-based mutation. Another mutation generation technique, DeepMutation [29], only supports three root causes in feed-forward neural networks. The limited mutation coverage of these techniques may result in datasets containing only a subset of DNN faults. DEFault addresses this limitation by creating an extended version of DeepCrime, allowing improved mutation coverage and enabling DEFault to be trained on a comprehensive set of possible faults and root causes.

Debugging DNN Programs: Recent research has proposed various techniques for diagnosing faults in DNN programs. Static analysis-based techniques, such as Neuralink [22], NerdBug [31], and DEBAR [20], analyze code structure and syntax without execution to detect faults, but may miss runtime-specific issues. On the other hand, dynamic analysis-based techniques, including UMLAUT [23], AutoTrainer [24], DeepDiagnosis [25], and DeepLocalize [21], use runtime information from the training session to detect faults. However, they rely on predefined rules that limit their ability to generalize. DeepFD [27] addresses these limitations by using dynamic features in ML classifiers for fault detection but overlooks structural model-related faults [15]. DEFault presents a *novel* approach that combines static and dynamic features in a hierarchical classification approach. It addresses the limitations of existing techniques by detecting, categorizing and root cause analyses of both training and model-related faults.

IX. Conclusion

In this paper, we present DEFault, a novel technique for detecting and diagnosing faults in DNN programs. Our experiments on a diverse dataset of 14,652 DNN programs and real-world faulty models from StackOverflow show that DEFault outperforms state-of-the-art techniques in fault detection and diagnosis of DNN programs. Moreover, the explainer module of DEFault effectively determines the root causes for model-related faults using static features. Our proposed dynamic features further enhance DEFault's performance in fault detection. Future work includes expanding the prototype to handle more architecture types (e.g., attention-based neural networks) and automatically fixing bugs.

References

[1] G. Montavon, W. Samek, and K.-R. Müller, "Methods for interpreting and understanding deep neural networks," *Digital signal processing*, vol. 73, pp. 1–15, 2018.

[2] A. M. Mubalaike and E. Adali, "Deep learning approach for intelligent financial fraud detection system," in *UBMK*, IEEE, 2018, pp. 598–603.

[3] P. Mahbub, O. Shuvo, and M. M. Rahman, "Explaining software bugs leveraging code structures in neural machine translation," in *Proceedings of 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 640–652.

[4] P. Mahbub and M. M. Rahman, "Predicting line-level defects by capturing code contexts with hierarchical transformers," in *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2024, pp. 308–319.

[5] A. Esteva, A. Robicquet, B. Ramsundar, *et al.*, "A guide to deep learning in healthcare," *Nature medicine*, vol. 25, no. 1, pp. 24–29, 2019.

[6] W. Liu, Y. Wen, Z. Yu, M. Li, B. Raj, and L. Song, "Sphereface: Deep hypersphere embedding for face recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 212–220.

[7] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, "A survey of deep learning techniques for autonomous driving," *Journal of field robotics*, vol. 37, no. 3, pp. 362–386, 2020.

[8] M. Dilhara, A. Ketkar, and D. Dig, "Understanding software-2.0: A study of machine learning library usage and evolution," *TOSEM*, vol. 30, no. 4, pp. 1–42, 2021.

[9] P. Devanbu, M. Dwyer, S. Elbaum, *et al.*, "Deep learning & software engineering: State of research and future directions," *arXiv preprint arXiv:2009.08525*, 2020.

[10] S. Jahan, M. B. Shah, and M. M. Rahman, "Towards understanding the challenges of bug localization in deep learning systems," *arXiv preprint arXiv:2402.01021*, 2024.

[11] H. L. Ribeiro, R. P. de Araujo, M. L. Chaim, H. A. de Souza, and F. Kon, "Jaguar: A spectrum-based fault localization tool for real-world software," in *ICS*, IEEE, 2018, pp. 404–409.

[12] P. Mahbub, *Comprehending software bugs leveraging code structures with neural language models.* Master's thesis, 2023.

[13] M. G. Abdolrasol, S. S. Hussain, T. S. Ustun, *et al.*, "Artificial neural networks based optimization techniques: A review," *Electronics*, vol. 10, no. 21, p. 2689, 2021.

[14] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang, "An empirical study on tensorflow program bugs," in *ISSTA*, 2018, pp. 129–140.

[15] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan, "A comprehensive study on deep learning bug characteristics," in *ESEC/FSE*, 2019, pp. 510–520.

[16] N. Humbatova, G. Jahangirova, G. Bavota, V. Riccio, A. Stocco, and P. Tonella, "Taxonomy of real faults in deep learning systems," in *ICSE*, 2020, pp. 1110–1121.

[17] H. Ben Braiek and F. Khomh, "Testing feedforward neural networks training programs," *TOSEM*, vol. 32, no. 4, pp. 1–61, 2023.

[18] M. M. Morovati, A. Nikanjam, F. Khomh, and Z. M. Jiang, "Bugs in machine learning-based systems: A faultload benchmark," *EMSE*, vol. 28, no. 3, p. 62, 2023.

[19] J. Chen, Y. Liang, Q. Shen, J. Jiang, and S. Li, "Toward understanding deep learning framework bugs," *TOSEM*, vol. 32, no. 6, pp. 1–31, 2023.

[20] Y. Zhang, L. Ren, L. Chen, Y. Xiong, S.-C. Cheung, and T. Xie, "Detecting numerical bugs in neural network architectures," in *ESEC/FSE*, 2020, pp. 826–837.

[21] M. Wardat, W. Le, and H. Rajan, "Deeplocalize: Fault localization for deep neural networks," in *ICSE*, IEEE, 2021, pp. 251–262.

[22] A. Nikanjam, H. B. Braiek, M. M. Morovati, and F. Khomh, "Automatic fault detection for deep learning programs using graph transformations," *TOSEM*, vol. 31, no. 1, pp. 1–27, 2021.

[23] E. Schoop, F. Huang, and B. Hartmann, "Umlaut: Debugging deep learning programs using program structure and model behavior," in *Proceedings of the 2021 CHI conference on human factors in computing systems*, 2021, pp. 1–16.

[24] X. Zhang, J. Zhai, S. Ma, and C. Shen, "Autotrainer: An automatic dnn training problem detection and repair system," in *ICSE*, IEEE, 2021, pp. 359–371.

[25] M. Wardat, B. D. Cruz, W. Le, and H. Rajan, "Deepdiagnosis: Automatically diagnosing faults and recommending actionable fixes in deep learning programs," in *ICSE*, 2022, pp. 561–572.

[26] N. Humbatova, G. Jahangirova, and P. Tonella, "Deepcrime: Mutation testing of deep learning systems based on real faults," in *ISSTA*, 2021, pp. 67–78.

[27] J. Cao, M. Li, X. Chen, *et al.*, "Deepfd: Automated fault diagnosis and localization for deep learning programs," in *ICSE*, 2022, pp. 573–585.

[28] H. V. Pham, T. Lutellier, W. Qi, and L. Tan, "Cradle: Cross-backend validation to detect and localize bugs in deep learning libraries," in *ICSE*, IEEE, 2019, pp. 1027–1038.

[29] L. Ma, F. Zhang, J. Sun, *et al.*, "Deepmutation: Mutation testing of deep learning systems," in *ISSRE*, IEEE, 2018, pp. 100–111.

[30] Anonymous, *Icse 2025 repository*, https://anonymous.4open.science/r/ICSE_2025-F42C, Accessed: 2024-11-29, 2025.

[31] F. Jafarinejad, K. Narasimhan, and M. Mezini, "Nerdbug: Automated bug detection in neural networks," in *Proceedings of the 1st ACM International Workshop on AI and Software Testing/Analysis*, 2021, pp. 13–16.

[32] D. Wu, B. Shen, Y. Chen, H. Jiang, and L. Qiao, "Tensfa: Detecting and repairing tensor shape faults in deep learning systems," in *ISSRE*, IEEE, 2021, pp. 11–21.

[33] S. Ma, Y. Liu, W.-C. Lee, X. Zhang, and A. Grama, "Mode: Automated neural network model debugging via state differential analysis and input selection," in *ESEC/FSE*, 2018, pp. 175–186.

[34] M. Yan, J. Chen, X. Zhang, L. Tan, G. Wang, and Z. Wang, "Exposing numerical bugs in deep learning via gradient back-propagation," in *ESEC/FSE*, 2021, pp. 627–638.

[35] A. Ghanbari, D.-G. Thomas, M. A. Arshad, and H. Rajan, "Mutation-based fault localization of deep neural networks," in *ASE*, Luxembourg, Luxembourg, 2023, pp. 1301–1313. DOI: 10.1109/ASE56229.2023.00171.

[36] H. F. Eniser, S. Gerasimou, and A. Sen, "Deepfault: Fault localization for deep neural networks," in *International Conference on Fundamental Approaches to Software Engineering*, Cham: Springer International Publishing, 2019, pp. 171–189.

[37] M. Wardat, B. D. Cruz, W. Le, and H. Rajan, "An effective data-driven approach for localizing deep learning faults," *arXiv preprint arXiv:2307.08947*, 2023.

[38] M. B. Shah, M. M. Rahman, and F. Khomh, "Towards enhancing the reproducibility of deep learning bugs: An empirical study," *Empirical Software Engineering*, vol. 30, no. 1, p. 23, 2025.

[39] S. Sun, "Meta-analysis of cohen's kappa," *Health Services and Outcomes Research Methodology*, vol. 11, pp. 145–163, 2011.

[40] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org.

[41] J. Johnson, *Convolutional Neural Networks for Visual Recognition*. Stanford University, Lecture Notes, 2017. [Online]. Available: http://cs231n.stanford.edu/.

[42] C. Ialongo, "Understanding the effect size and its measures.," *Biochemia medica*, vol. 26, no. 2, pp. 150–163, 2016.

[43] D. K. Lee, "Alternatives to p value: Confidence interval and effect size," *Korean journal of anesthesiology*, vol. 69, no. 6, pp. 555–562, 2016.

[44] Q. Hu, L. Ma, X. Xie, B. Yu, Y. Liu, and J. Zhao, "Deepmutation++: A mutation testing framework for deep learning systems," in *ASE*, IEEE, 2019, pp. 1158–1161.

[45] Q. Zhu, A. Zaidman, and A. Panichella, "How to kill them all: An exploratory study on the impact of code observability on mutation testing," *JSS*, vol. 173, p. 110 864, 2021.

[46] A. Skabar, "Automatic mlp weight regularization on mineralization prediction tasks," in *Knowledge-Based Intelligent Information and Engineering Systems: 9th International Conference, KES 2005, Melbourne, Australia, September 14-16, 2005, Proceedings, Part III*, vol. 9, Springer Berlin Heidelberg, 2005. DOI: 10.1007/11553939_2.

[47] M. S. Advani, A. M. Saxe, and H. Sompolinsky, "High-dimensional dynamics of generalization error in neural networks," *Neural Networks*, vol. 132, pp. 428–446, 2020. DOI: 10.1016/j.neunet.2020.09.010.

[48] X. Hu, L. Liang, S. Li, *et al.*, "Deepsniffer: A dnn model extraction framework based on learning architectural hints," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 385–399.

[49] Q. Shen, H. Ma, J. Chen, Y. Tian, S.-C. Cheung, and X. Chen, "A comprehensive study of deep learning compiler bugs," in *ESEC/FSE*, 2021, pp. 968–980.

[50] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning (still) requires rethinking generalization," *Communications of the ACM*, vol. 64, no. 3, pp. 107–115, 2021.

[51] A. Nguyen, J. Yosinski, and J. Clune, "Understanding neural networks via feature visualization: A survey," *Explainable AI: interpreting, explaining and visualizing deep learning*, pp. 55–76, 2019.

[52] Y. Bengio, "Practical recommendations for gradient-based training of deep architectures," in *Neural networks: Tricks of the trade: Second edition*, Springer, 2012, pp. 437–478.

[53] L. N. Smith, "Cyclical learning rates for training neural networks," in *WACV*, IEEE, 2017, pp. 464–472.

[54] N. Pinto, D. Doukhan, J. J. DiCarlo, and D. D. Cox, "A high-throughput screening approach to discovering good forms of biologically inspired visual representation," *PLoS computational biology*, vol. 5, no. 11, e1000579, 2009.

[55] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, JMLR Workshop and Conference Proceedings, 2010, pp. 249–256.

[56] P. Arabie, L. Hubert, G. De Soete, and A. Gordon, "Hierarchical classification," *P. Arabie, L. Hubert, G. De Soete, & A. Gordon, Clustering and classification*, pp. 65–121, 1996.

[57] S. J. Rigatti, "Random forest," *Journal of Insurance Medicine*, vol. 47, no. 1, pp. 31–39, 2017.

[58] L. Breiman, "Random forests," *Machine learning*, vol. 45, pp. 5–32, 2001.

[59] D. A. Anggoro and S. S. Mukti, "Performance comparison of grid search and random search methods for hyperparameter tuning in extreme gradient boosting algorithm to predict chronic kidney failure.," *International Journal of Intelligent Engineering & Systems*, vol. 14, no. 6, 2021.

[60] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization.," *JMLR*, vol. 13, no. 2, 2012.

[61] P. P. Angelov, E. A. Soares, R. Jiang, N. I. Arnold, and P. M. Atkinson, "Explainable artificial intelligence: An analytical review," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 11, no. 5, e1424, 2021.

[62] S. Mangalathu, S.-H. Hwang, and J.-S. Jeon, "Failure mode and effects analysis of rc members based on machine-learning-based shapley additive explanations (shap) approach," *Engineering Structures*, vol. 219, p. 110 927, 2020.

[63] N. W. S. Wardhani, M. Y. Rochayani, A. Iriany, A. D. Sulistyono, and P. Lestantyo, "Cross-validation metrics for evaluating classification performance on imbalanced data," in *IC3INA*, IEEE, 2019, pp. 14–18.

[64] Q. Zheng, X. Tian, N. Jiang, and M. Yang, "Layer-wise learning based stochastic gradient descent method for the optimization of deep convolutional neural network," *Journal of Intelligent & Fuzzy Systems*, vol. 37, no. 4, pp. 5641–5654, 2019.

[65] X. Wang, Y. Qin, Y. Wang, S. Xiang, and H. Chen, "Reltanh: An activation function with vanishing gradient resistance for sae-based dnns and its application to rotating machinery fault diagnosis," *Neurocomputing*, vol. 363, pp. 88–98, 2019.

[66] G. H. Dunteman and M.-H. R. Ho, *An introduction to generalized linear models*. Sage Publications, 2005.

[67] S. M. Lundberg, G. Erion, H. Chen, *et al.*, "From local explanations to global understanding with explainable ai for trees," *Nature machine intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[68] B. Liu, C. Wen, A. D. Sarwate, and M. M. Dehnavi, "A unified optimization approach for sparse tensor operations on gpus," in *CLUSTER*, IEEE, 2017, pp. 47–57.

[69] D. R. Wilson and T. R. Martinez, "The general inefficiency of batch training for gradient descent learning," *Neural networks*, vol. 16, no. 10, pp. 1429–1451, 2003.

[70] S. Ghalebikesabi, L. Ter-Minassian, K. DiazOrdaz, and C. C. Holmes, "On locality of local explanation models," *Advances in neural information processing systems*, vol. 34, pp. 18 395–18 407, 2021.

[71] S. Technologies, *Tensorflow 2 published models*, https://github.com/sarus-tech/tf2-published-models, Accessed: 2024-11-21, 2023.

[72] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[73] S. S. Kadam, A. C. Adamuthe, and A. B. Patil, "Cnn model for image classification on mnist and fashion-mnist dataset," *Journal of scientific research*, vol. 64, no. 2, pp. 374–384, 2020.

[74] S. Bruch, X. Wang, M. Bendersky, and M. Najork, "An analysis of the softmax cross entropy loss for learning-to-rank with binary relevance," in *Proceedings of the 2019 ACM SIGIR international conference on theory of information retrieval*, 2019, pp. 75–78.

[75] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.

[76] G. T. Smith, "On construct validity: Issues of method and measurement.," *Psychological assessment*, vol. 17, no. 4, p. 396, 2005.

[77] M. G. Findley, K. Kikuta, and M. Denly, "External validity," *Annual Review of Political Science*, vol. 24, no. 1, pp. 365–393, 2021.